

用兼容类创建动态快捷方式

这段代码是**兼容低版本的动态快捷方式创建逻辑**，核心目的是在不同 Android 版本的手机上，安全地给 App 添加“桌面长按快捷方式”，同时避免因系统不支持而崩溃。咱们用大白话结合代码细节，让小白也能吃透～

先明确核心作用

和之前的动态快捷方式一样，最终效果是用户长按 App 图标时，能看到“First Shortcut”和“- Second Shortcut”两个快捷入口，点击直接打开对应的页面。但这段代码更“贴心”——会先检查手机是否支持这个功能，不支持就提示用户，避免 App 崩溃。

逐行拆解（小白友好版）

```
private void createDynamicShortcuts() {  
    // 第一步：先检查手机是否能支持快捷方式（关键！）  
    // 条件1：系统版本 >= Android 7.1（API 25），因为7.1才开始支持快捷方式  
    // 条件2：通过兼容类判断手机是否真的能创建快捷方式（有些手机系统可能阉割了这个功能）  
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.N_MR1  
        && ShortcutManagerCompat.isRequestPinShortcutSupported(this)) {  
        // 第二步：创建第一个快捷方式（对应StringActivity页面）  
        // 1. 定义点击后要打开的页面：StringActivity  
        Intent stringIntent = new Intent(this, StringActivity.class);  
        stringIntent.setAction(Intent.ACTION_VIEW); // 固定写法，告诉系统“打开页面”  
        // 2. 给快捷方式设置“名字、图标、目标页面”  
        ShortcutInfoCompat firstShortcut = new ShortcutInfoCompat.Builder(this, "first_dynamic  
)  
            .setShortLabel("First Shortcut") // 短名称（长按图标时显示）  
            .setLongLabel("Open String Activity") // 长名称（空间够时显示）  
            .setIcon(IconCompat.createWithResource(this, R.mipmap.ic_launcher)) // 图标  
            .setIntent(stringIntent) // 绑定点击后要打开的页面  
            .build(); // 构建完成  
        // 第三步：创建第二个快捷方式（对应LoginInputActivity页面）  
        // 和第一个步骤完全一样，只是目标页面换成了LoginInputActivity  
        Intent loginIntent = new Intent(this, LoginInputActivity.class);  
        loginIntent.setAction(Intent.ACTION_VIEW);  
        ShortcutInfoCompat secondShortcut = new ShortcutInfoCompat.Builder(this, "second_  
dynamic")
```

```

        .setShortLabel("Second Shortcut")
        .setLongLabel("Open Login Activity")
        .setIcon(IconCompat.createWithResource(this, R.mipmap.ic_launcher))
        .setIntent(loginIntent)
        .build();
// 第四步：把两个快捷方式添加到系统中
// 用兼容类ShortcutManagerCompat的方法，自动适配不同系统版本
ShortcutManagerCompat.addDynamicShortcuts(this,
        Arrays.asList(firstShortcut, secondShortcut));
// 提示用户：快捷方式创建成功
Toast.makeText(this, "快捷方式已创建，请长按应用图标查看", Toast.LENGTH_SHORT).show
());
    } else {
        // 如果手机不支持（比如系统版本太低），提示用户
        Toast.makeText(this, "当前设备不支持快捷方式功能", Toast.LENGTH_SHORT).show();
    }
}
}

```

关键技术点：“兼容性”是核心

这段代码的精髓在于**对不同手机系统的兼容性处理**，咱们重点讲两个兼容类：

1. ShortcutManagerCompat（快捷方式兼容管理器）

- 为什么用它？

原生的 `ShortcutManager` 只支持 Android 7.1（API 25）及以上，而 `ShortcutManagerCompat` 是 AndroidX 库提供的兼容类，能自动适配低版本系统（比如在 7.0 及以下手机上，会安全地不执行操作，避免崩溃）。

- 核心方法：
 - `isRequestPinShortcutSupported(this)`：检查当前手机是否支持快捷方式功能（比单纯判断系统版本更靠谱，因为有些手机厂商会阉割功能）。
 - `addDynamicShortcuts(...)`：添加动态快捷方式，内部会处理不同系统的差异。

2. ShortcutInfoCompat 和 IconCompat

- 这两个也是兼容类，对应原生的 `ShortcutInfo` 和 `Icon`。
- 作用：在低版本手机上，即使不支持快捷方式，用它们创建对象也不会报错（原生类在低版本上会直接崩溃）。

和之前代码的区别：更安全、更通用

之前的代码（原生类）	这段代码（兼容类）
用 <code>ShortcutManager</code> 和 <code>ShortcutInfo</code>	用 <code>ShortcutManagerCompat</code> 和 <code>ShortcutInfoCompat</code>
只支持 Android 7.1 及以上	自动适配所有系统版本（低版本不崩溃）
需手动判断系统版本	兼容类内部已处理版本判断

实际开发中的使用场景

- 当你的 App 需要适配老手机（比如 Android 6.0、7.0）时，必须用这种兼容写法，否则在老手机上打开会崩溃。
- 比如微信、支付宝这样的国民 App，用户手机型号五花八门，肯定要用兼容类确保所有用户都能正常使用（不支持的用户会看到提示，而不是 App 崩溃）。

小白总结

这段代码的核心是：**用兼容类安全地创建动态快捷方式，先检查手机是否支持，支持就创建，不支持就提示。**

- 关键是那两个兼容类：`ShortcutManagerCompat`（管适配）和 `ShortcutInfoCompat`（管创建快捷方式）。
 - 记住：做 App 开发时，一定要考虑老手机的兼容性，否则会有用户用不了你的 App 哦～
- （注：文档部分内容可能由 AI 生成）